Block 1 — Install

```
# 1. Install core Python libraries
!pip install pymupdf pytesseract jiwer google-generativeai pillow opencv-python-headless python-docx textblob -

# 2. Install Tesseract OS-level dependencies
!apt-get install -y tesseract-ocr poppler-utils -q

# 3. Download your secret weapon: The 17th-century Spanish model
!wget -q https://github.com/tesseract-ocr/tessdata/raw/main/spa_old.traineddata -P /usr/share/tesseract-ocr/4.0
!wget https://github.com/Kunsh1/RenAIssance/releases/download/Text_Scan_pdfs/Text.Scans.zip
!mv Text.Scans.zip "Text Scans.zip"
# Note: We will also need to upload "Transcription.zip" in the colab env
!pip install groq -q
# 4. Install NLTK data for your text evaluation
import nltk
import subprocess

# Quiet downloads for NLTK
nltk.download('punkt_tab', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)

# Quiet TextBlob corpora download
subprocess.run(
    ['python', '-m', 'textblob.download_corpora', '-q'],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL
)
print("Installations complete! 🚀")
```

```
# Unzip Transcription and Text Scan Folder
# ----------------------------------------

import zipfile
import os

zip_files = ["Text Scans.zip", "Transcription.zip"]

for zip_path in zip_files:
    if not os.path.exists(zip_path):
        print(f"❌ File not found: {zip_path}")
        continue

    extract_folder = zip_path.replace(".zip", "")

    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_folder)

    print(f"✅ Extracted: {zip_path} → {extract_folder}")
```

```
✅ Extracted: Text Scans.zip → Text Scans
✅ Extracted: Transcription.zip → Transcription
```

Block 2 — Imports

```
import fitz
import cv2
import numpy as np
import pytesseract
from groq import Groq
from PIL import Image
import gc
from jiwer import cer, wer
import os, re, time, math, collections
from docx import Document
from textblob import TextBlob
# Disable Pillow RAM limits to prevent "DecompressionBomb" errors on large PDFs
Image.MAX_IMAGE_PIXELS = None
```

```python

from google.colab import userdata
GROQ_API_KEY = userdata.get('Grok_api')   # add this key name in Colab Secrets
GROQ_MODEL   = "llama-3.3-70b-versatile"
groq_client  = Groq(api_key=GROQ_API_KEY)
BATCH_SIZE = 3
print("Imports loaded successfully!")
```

```
Imports loaded successfully!
```

Block A — Docx Ground Truth Parser

```python
# Keep only the fallback default — no manual per-file entries needed
DEFAULT_MARKER_FORMAT = r"PDF\s*p(\d+)"

# All known marker patterns to try, in order of specificity
# Add new ones here only if a completely new format appears
MARKER_CANDIDATES = [
    # "PDF p2 - left" / "PDF p2 – right"
    r"PDF\s*p(\d+)\s*[--]\s*(left|right)",
    # "PDF p2"
    r"PDF\s*p(\d+)",
]


def detect_marker_format(doc) -> str:
    """
    Scan first 30 paragraphs of a docx to auto-detect which
    page marker pattern is in use. Returns the matching pattern.
    Falls back to DEFAULT_MARKER_FORMAT if nothing matches.
    """
    for para in doc.paragraphs[:30]:
        text = para.text.strip()
        if not text:
            continue
        for pattern in MARKER_CANDIDATES:
            if re.match(pattern, text, re.IGNORECASE):
                print(f"  Auto-detected marker format: '{pattern}' from '{text}'")
                return pattern

    print(f"  No marker detected in first 30 paragraphs, using default.")
    return DEFAULT_MARKER_FORMAT


def parse_docx_ground_truth(docx_path: str) -> dict:
    if not docx_path:
        print("No transcription docx provided — skipping.")
        return {}
    if not os.path.exists(docx_path):
        print(f"WARNING: '{docx_path}' not found — skipping.")
        return {}

    doc            = Document(docx_path)
    marker_pattern = detect_marker_format(doc)   # ← auto detected here

    pages        = {}
    current_page = None
    buffer       = []

    for para in doc.paragraphs:
        text = para.text.strip()
        if not text:
            if current_page is not None:
                buffer.append("")
            continue

        is_note = any(run.font.highlight_color is not None for run in para.runs)
        if is_note:
            continue

        match = re.match(marker_pattern, text, re.IGNORECASE)
        if match:
            if current_page is not None:
```

```
                pages[current_page] = "\n".join(buffer).strip()

            page_num = int(match.group(1))
            # If pattern captured a side (left/right), include it in key
            if match.lastindex >= 2:
                side = match.group(2).lower().strip()
                current_page = f"{page_num}_{side}"
            else:
                current_page = page_num

            buffer = []
            continue

        if current_page is not None:
            buffer.append(text)

    if current_page is not None and buffer:
        pages[current_page] = "\n".join(buffer).strip()

    docx_name = os.path.basename(docx_path)
    print(f"  Parsed '{docx_name}': pages {sorted(str(k) for k in pages.keys())}")
    return pages


def get_ground_truth_for_range(docx_path: str, start_page: int, end_page: int) -> dict:
    """
    Returns ground truth only for pages in [start_page, end_page].
    If end_page is 0, returns all available pages from start_page onwards.
    """
    all_pages = parse_docx_ground_truth(docx_path)
    return {
        p: text for p, text in all_pages.items()
        if p >= start_page and (end_page == 0 or p <= end_page)
    }
```

Block B — Main Config + Runner

```
# ── FOLDER PATHS ──────────────────────────────────────
PDF_DIR   = "Text Scans/"
DOCX_DIR  = "Transcription/"
# ──────────────────────────────────────────────────────

SOURCES = [
    {
        "pdf_path"  : os.path.join(PDF_DIR,  "Buendia - Instruccion.pdf"),
        "docx_path" : os.path.join(DOCX_DIR, "Buendia - Instruccion transcription.docx"),
        "start_page": 2,
        "end_page"  : 4,
    },
    {
        "pdf_path"  : os.path.join(PDF_DIR,  "Covarrubias - Tesoro lengua.pdf"),
        "docx_path" : os.path.join(DOCX_DIR, "Covarrubias - Tesoro lengua transcription.docx"),
        "start_page": 7,
        "end_page"  : 9,
    },
    {
        "pdf_path"  : os.path.join(PDF_DIR,  "Guardiola - Tratado nobleza.pdf"),
        "docx_path" : os.path.join(DOCX_DIR, "Guardiola - Tratado nobleza transcription.docx"),
        "start_page": 12,
        "end_page"  : 14,
    },
    {
        "pdf_path"  : os.path.join(PDF_DIR,  "PORCONES.228.38 – 1646.pdf"),
        "docx_path" : os.path.join(DOCX_DIR, "PORCONES.228.38 - 1646 transcription.docx"),
        "start_page": 1,
        "end_page"  : 5,
    },
    {
        "pdf_path"  : os.path.join(PDF_DIR,  "PORCONES.23.5 - 1628.pdf"),
        "docx_path" : os.path.join(DOCX_DIR, "PORCONES.23.5 - 1628 transcription.docx"),
        "start_page": 1,
        "end_page"  : 4,
    },
```

```
    {
        "pdf_path"  : os.path.join(PDF_DIR,  "PORCONES.748.6 – 1650.pdf"),
        "docx_path" : os.path.join(DOCX_DIR, "PORCONES.748.6 – 1650 Transcription.docx"),
        "start_page": 2,
        "end_page"  : 4,
    },
]

# Quick sanity check before running anything
for source in SOURCES:
    pdf_ok  = os.path.exists(source["pdf_path"])
    docx_ok = os.path.exists(source["docx_path"]) if source["docx_path"] else "skipped"
    print(f"{os.path.basename(source['pdf_path']):<45} PDF: {str(pdf_ok):<6} DOCX: {docx_ok}")
```

```
Buendia - Instruccion.pdf                     PDF: True   DOCX: True
Covarrubias - Tesoro lengua.pdf               PDF: True   DOCX: True
Guardiola - Tratado nobleza.pdf               PDF: True   DOCX: True
PORCONES.228.38 – 1646.pdf                    PDF: True   DOCX: True
PORCONES.23.5 - 1628.pdf                      PDF: True   DOCX: True
PORCONES.748.6 – 1650.pdf                     PDF: True   DOCX: True
```

```
# ── OCR PIPELINE ──────────────────────────────────────────
def estimate_raw_quality(text):
    """
    Rough quality estimate based on ratio of common Spanish function words.
    Returns a score 0-1. Above 0.15 means OCR is already clean enough to skip Groq.
    """
    COMMON_SPANISH = {
        'de','la','el','en','y','a','que','los','las','un','una',
        'con','por','del','se','su','al','es','no','le','lo',
        'como','mas','pero','si','sobre','entre','cuando','muy'
    }
    words = [w.lower().strip('.,;:()[]') for w in text.split()]
    if not words:
        return 0.0
    hits = sum(1 for w in words if w in COMMON_SPANISH)
    return hits / len(words)


def process_source(pdf_path, docx_path=None, start_page=2, end_page=10):
    """
    Runs ONLY the OCR pipeline for one source.
    Evaluation is a separate step — call evaluate_source() independently.
    Returns raw_results and cleaned_results only.
    """
    source_name = os.path.splitext(os.path.basename(pdf_path))[0]
    output_dir  = f"outputs/{source_name}/"
    pages_dir   = f"pages/{source_name}/"
    os.makedirs(output_dir, exist_ok=True)

    print(f"\n{'='*60}")
    print(f"SOURCE : {source_name}")
    print(f"Pages  : {start_page} → {'END' if end_page == 0 else end_page}")
    print(f"{'='*60}")

    # Step 1: PDF → Images
    image_paths = pdf_to_images(
        pdf_path, pages_dir,
        start_page=start_page,
        end_page=end_page
    )
    if not image_paths:
        print(f"No images extracted for {source_name}, skipping.")
        return None

    # Step 2: OCR
    raw_results = {}
    for path in image_paths:
        print(f"  OCR: {os.path.basename(path)}")
        raw = ocr_page(path)
        if raw.strip():
            raw_results[path] = rule_based_fix(raw)
        else:
            print(f"  WARNING: No text from {path}")

    print(f"\n  OCR complete — {len(raw_results)} pages.")
```

```python

    # Step 3: Groq batch cleaning with quality gate
    cleaned_results = {}
    pages_list = list(raw_results.items())

    for batch_start in range(0, len(pages_list), BATCH_SIZE):
        batch = pages_list[batch_start: batch_start + BATCH_SIZE]

        # Quality gate — check average raw quality of this batch
        avg_quality = sum(estimate_raw_quality(t) for _, t in batch) / len(batch)

        batch_dict = {
            os.path.basename(p).replace(".png", ""): t
            for p, t in batch
        }

        if avg_quality >= 0.23:
            # Raw OCR is already decent — skip Groq
            print(f"  Batch {batch_start//BATCH_SIZE + 1}: quality {avg_quality:.2f} — skipping Groq, OCR alre
            cleaned_batch = {pid: txt for pid, txt in batch_dict.items()}
        else:
            # Raw OCR is poor — send to Groq
            print(f"  Groq batch {batch_start//BATCH_SIZE + 1}: quality {avg_quality:.2f} — {list(batch_dict.k
            cleaned_batch = clean_ocr_batch(batch_dict)
            time.sleep(4)

        for (path, _), (page_id, cleaned_text) in zip(batch, cleaned_batch.items()):
            cleaned_results[path] = cleaned_text
            with open(f"{output_dir}{page_id}_raw.txt", "w", encoding="utf-8") as f:
                f.write(raw_results[path])
            with open(f"{output_dir}{page_id}_cleaned.txt", "w", encoding="utf-8") as f:
                f.write(cleaned_text)

    print(f"  OCR outputs saved to {output_dir}")
    return {"raw": raw_results, "cleaned": cleaned_results}


# ── EVALUATION PIPELINE ──────────────────────────────────

def evaluate_source(ocr_result, docx_path, start_page=2,
                    end_page=10, source_name="source"):
    """
    Runs ONLY evaluation for one source.
    Call this independently after process_source().

    ocr_result : dict returned by process_source()
    docx_path  : transcription .docx path, or None to skip eval
    """
    if ocr_result is None:
        print("No OCR result to evaluate.")
        return

    if not docx_path:
        print(f"No docx for {source_name} — evaluation skipped.")
        return

    output_dir = f"outputs/{source_name}/"
    ground_truth = get_ground_truth_for_range(docx_path, start_page, end_page)

    if not ground_truth:
        print(f"No ground truth pages found in range {start_page}–{end_page}.")
        return

    evaluate(
        raw_results     = ocr_result["raw"],
        cleaned_results = ocr_result["cleaned"],
        ground_truth    = ground_truth,
        output_dir      = output_dir,
        source_name     = source_name,
        start_page      = start_page
    )


# ── TOP LEVEL RUNNERS ────────────────────────────────────

# CHANGE: replace the entire run_ocr_all function
```

```python

def run_ocr_all(sources):
    all_results = {}

    if isinstance(sources, dict):
        iterator = sources.items()
    elif isinstance(sources, list) and len(sources) > 0 and isinstance(sources[0], tuple):
        iterator = sources
    elif isinstance(sources, list) and len(sources) > 0 and isinstance(sources[0], dict):
        iterator = []
        for s in sources:
            name = s.get('name', s.get('pdf', s.get('pdf_path', 'Unknown_Source')))
            iterator.append((name, s))
    else:
        print(f"❌ ERROR: Unrecognised SOURCES format: {sources}")
        return {}

    for source_name, source_info in iterator:
        pdf_path  = source_info.get('pdf', source_info.get('pdf_path'))
        docx_path = source_info.get('docx', source_info.get('docx_path'))
        start_p   = source_info.get('start', source_info.get('start_page', 1))
        end_p     = source_info.get('end', source_info.get('end_page', 999))

        if not pdf_path:
            print(f"  [Warning] Skipping {source_name} — no PDF path found.")
            continue

        result = process_source(pdf_path, docx_path, start_p, end_p)  # FIX 3: don't unpack as tuple

        if result:
            # FIX 1: use 'cleaned' not 'clean'
            # FIX 2: key by pdf_path so run_eval_all can find it
            all_results[pdf_path] = {
                'raw':     result['raw'],
                'cleaned': result['cleaned'],
            }

    print("\nOCR complete for all sources.")
    return all_results


def run_eval_all(all_results, sources=SOURCES):
    """Run evaluation for all sources that have a docx. Call after run_ocr_all()."""
    for source in sources:
        pdf_path = source["pdf_path"]
        docx_path = source.get("docx_path")
        source_name = os.path.splitext(os.path.basename(pdf_path))[0]

        evaluate_source(
            ocr_result  = all_results.get(pdf_path),
            docx_path   = docx_path,
            start_page  = source["start_page"],
            end_page    = source["end_page"],
            source_name = source_name
        )
```

Block 3 — PDF to Images

```python
def is_double_spread(width, height, threshold=1.5):
    """
    Automatically detect if a scanned page is a double-page spread.
    threshold: width/height ratio above which we consider it a spread.
    1.5 is safe — a single page is ~0.7, a double spread is ~1.4+
    """
    return (width / height) >= threshold


def pdf_to_images(pdf_path, output_dir, dpi=300,
                  start_page=2, end_page=10):
    os.makedirs(output_dir, exist_ok=True)
    doc = fitz.open(pdf_path)
    image_paths = []

```

```
    for i, page in enumerate(doc):
        page_num = i + 1
        if page_num < start_page:
            continue
        if end_page != 0 and page_num > end_page:
            break

        try:
            # Base zoom calculation
            zoom = dpi / 72.0

            # --- THE SAFETY CAP ---
            # Check the physical size of the page first.
            # If the calculated zoom makes it larger than 4000px on its longest edge, scale it down.
            max_dimension = max(page.rect.width, page.rect.height) * zoom
            if max_dimension > 4000:
                zoom = 4000 / max(page.rect.width, page.rect.height)
                print(f"  Page {page_num} is too large. Scaling down to safe RAM limits.")
            # ---------------------

            mat = fitz.Matrix(zoom, zoom)
            pix = page.get_pixmap(matrix=mat, clip=page.rect)
            img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)

            if is_double_spread(pix.width, pix.height):
                mid   = pix.width // 2
                left  = img.crop((0, 0, mid, pix.height))
                right = img.crop((mid, 0, pix.width, pix.height))

                left_path  = os.path.join(output_dir, f"page_{page_num:03d}_left.png")
                right_path = os.path.join(output_dir, f"page_{page_num:03d}_right.png")

                left.save(left_path)
                right.save(right_path)
                image_paths.extend([left_path, right_path])
                print(f"  Page {page_num} → double spread, split into left + right")

            else:
                out_path = os.path.join(output_dir, f"page_{page_num:03d}.png")
                img.save(out_path)
                image_paths.append(out_path)
                print(f"  Page {page_num} → single page")

            # --- FORCE CLEAR RAM IMMEDIATELY ---
            del img
            if is_double_spread(pix.width, pix.height):
                del left, right
            # ----------------------------------

        except Exception as e:
            print(f"  ERROR page {page_num}: {e}")
        finally:
            del pix

    doc.close()
    return image_paths
```

Block 4 — Tesseract OCR Per Region

```
def get_main_text_crop(image_path):
    """Uses Computer Vision to find the main text block and ignore marginalia."""
    # 1. Read the image and convert to grayscale
    img = cv2.imread(image_path)
    original = img.copy()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 2. Binarize the image (makes ink pure black, paper pure white)
    _, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    # 3. Dilate the ink to connect words and lines into solid blocks
    # A kernel of (50, 10) connects horizontal text well without bridging the gap to marginalia
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (50, 10))
    dilated = cv2.dilate(thresh, kernel, iterations=2)
```

```python

    # 4. Find the contours (the boundaries of these blocks)
    contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    height, width = img.shape[:2]
    main_text_boxes = []

    # 5. Filter the blocks to find the "Main Text"
    for c in contours:
        x, y, w, h = cv2.boundingRect(c)
        area = w * h

        # CRITERIA TO IGNORE MARGINALIA & EMBELLISHMENTS:
        # - Ignore tiny blocks (dust, page numbers)
        # - Ignore narrow blocks (marginalia on the far edges)
        # - The main text usually spans at least 40% of the page width
        if area > (height * width * 0.02) and w > (width * 0.40):
            main_text_boxes.append((x, y, w, h))

    # If it fails to find a distinct block, return the whole image as a fallback
    if not main_text_boxes:
        print(f"  [Layout Warning] Could not isolate main text on {os.path.basename(image_path)}.")
        return Image.fromarray(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))

    # 6. Find the extreme boundaries that contain ALL the main text paragraphs
    x_min = min([b[0] for b in main_text_boxes])
    y_min = min([b[1] for b in main_text_boxes])
    x_max = max([b[0] + b[2] for b in main_text_boxes])
    y_max = max([b[1] + b[3] for b in main_text_boxes])

    # Add a tiny 10-pixel padding around the final box so we don't cut off letters
    x_min, y_min = max(0, x_min - 10), max(0, y_min - 10)
    x_max, y_max = min(width, x_max + 10), min(height, y_max + 10)

    # 7. Crop the image!
    cropped_img = original[y_min:y_max, x_min:x_max]

    # Convert back to PIL Image for Tesseract
    return Image.fromarray(cv2.cvtColor(cropped_img, cv2.COLOR_BGR2RGB))


def preprocess_for_ocr(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # ADD: upscale if image is small — Tesseract works best at 300dpi+
    h, w = gray.shape
    if w < 2000:
        scale = 2000 / w
        gray = cv2.resize(gray, None, fx=scale, fy=scale, interpolation=cv2.INTER_CUBIC)

    # EXISTING: denoise
    denoised = cv2.fastNlMeansDenoising(gray, h=30)

    # EXISTING: CLAHE contrast
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(denoised)

    # EXISTING: Otsu binarize
    _, binary = cv2.threshold(enhanced, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    return Image.fromarray(binary)


def ocr_page(image_path):
    try:
        preprocessed = preprocess_for_ocr(image_path)
        custom_config = r'--oem 1 --psm 4'
        raw_text = pytesseract.image_to_string(preprocessed, lang='spa_old', config=custom_config)
        preprocessed.close()
        del preprocessed
        gc.collect()
        return raw_text.strip()
    except Exception as e:
        print(f"  OCR failed for {image_path}: {e}")
        return ""
```

Block 5 — Rule-Based Pre-Fix (Before LLM)

```python
def rule_based_fix(text):
    # EXISTING: "fs" → "ss"
    text = re.sub(r'fs', 'ss', text)

    # EXISTING: word-final 'f' → 's'
    text = re.sub(r'f\b', 's', text)

    # ADD: Unicode long-s character ſ (U+017F) → 's'
    text = text.replace('ſ', 's')

    # ADD: common Tesseract ligature noise
    text = text.replace('ﬁ', 'fi')
    text = text.replace('ﬂ', 'fl')
    text = text.replace('ﬀ', 'ff')
    text = text.replace('ﬃ', 'ffi')

    # ADD: strip lines that are pure noise (less than 2 alphabetic chars)
    lines = text.splitlines()
    lines = [l for l in lines if sum(c.isalpha() for c in l) >= 2]
    text = '\n'.join(lines)

    return text
```

Block 6 — Batched Groq

```python
import time
import re

def clean_ocr_batch(page_texts: dict, max_retries: int = 3) -> dict:
    cleaned_pages = {}

    non_empty = {pid: txt for pid, txt in page_texts.items() if txt.strip()}
    for pid in page_texts:
        if pid not in non_empty:
            cleaned_pages[pid] = ""

    if not non_empty:
        return cleaned_pages

    pages_block = "\n\n".join(f"[PAGE: {pid}]\n{txt}" for pid, txt in non_empty.items())

    for attempt in range(1, max_retries + 1):
        print(f"  Groq attempt {attempt}/{max_retries} — batch of {len(non_empty)} pages...")
        try:
            response = groq_client.chat.completions.create(
                model=GROQ_MODEL,
                messages=[
                    {
                        "role": "system",
                        "content": (
                            "You are an expert in 17th-century Spanish palaeography. "
                            "For EACH page block marked [PAGE: <id>]: fix long-s errors ('f' used as 's'), "
                            "correct OCR noise, preserve archaic spellings. "
                            "Return ONLY the cleaned text keeping the exact [PAGE: <id>] markers. "
                            "No explanations, no preamble."
                        )
                    },
                    {"role": "user", "content": pages_block}
                ],
                temperature=0.1,
                max_tokens=8192,
                frequency_penalty=1.0,   # ADD
                presence_penalty=1.0,
            )
            raw_response = response.choices[0].message.content.strip()
            break  # success

```

```
        except Exception as e:
            error_str = str(e)
            if "429" in error_str or "rate" in error_str.lower():
                wait_match = re.search(r'try again in ([\d\.]+)s', error_str, re.IGNORECASE)
                wait_time  = float(wait_match.group(1)) + 2.0 if wait_match else 60.0
                print(f"  ⏳ Rate limited — sleeping {wait_time:.0f}s (attempt {attempt}/{max_retries})")
                time.sleep(wait_time)
            else:
                print(f"  ❌ Groq error: {error_str} — falling back to rule_based_fix.")
                for pid, txt in non_empty.items():
                    cleaned_pages[pid] = rule_based_fix(txt)
                return cleaned_pages

            if attempt == max_retries:
                print("  ❌ Max retries reached — falling back to rule_based_fix.")
                for pid, txt in non_empty.items():
                    cleaned_pages[pid] = rule_based_fix(txt)
                return cleaned_pages

    # Parse [PAGE: id] markers out of the response
    sections = re.split(r'\[PAGE:\s*([^\]]+)\]', raw_response)
    parsed = {}
    for i in range(1, len(sections) - 1, 2):
        parsed[sections[i].strip()] = sections[i + 1].strip()

    for pid, original in non_empty.items():
        if pid in parsed and parsed[pid]:
            cleaned_pages[pid] = parsed[pid]
            print(f"    ✅ {pid}: cleaned ({len(parsed[pid])} chars)")
        else:
            print(f"    ⚠️  {pid}: missing from response — using rule_based_fix")
            cleaned_pages[pid] = rule_based_fix(original)

    return cleaned_pages
```

Block 7 — Evaluation

```
def normalize_for_eval(text: str) -> str:
    """Normalize per ground truth rules before any metric computation."""
    text = text.lower().strip()
    text = text.replace('v', 'u')
    text = text.replace('ç', 'z')
    text = re.sub(r'\s+', ' ', text)
    return text

# ── TextBlob Metric Helpers ──────────────────────────────

def ngram_precision(reference: str, hypothesis: str, n: int) -> float:
    """Precision of n-grams in hypothesis vs reference (BLEU-like)."""
    ref_ngrams = collections.Counter(
        tuple(ng) for ng in TextBlob(reference).ngrams(n)
    )
    hyp_ngrams = collections.Counter(
        tuple(ng) for ng in TextBlob(hypothesis).ngrams(n)
    )
    if not hyp_ngrams:
        return 0.0
    matches = sum(min(hyp_ngrams[ng], ref_ngrams[ng]) for ng in hyp_ngrams)
    return matches / sum(hyp_ngrams.values())


def bleu_score(reference: str, hypothesis: str, max_n: int = 4) -> float:
    """
    Corpus BLEU approximation using TextBlob n-grams.
    Geometric mean of 1..max_n gram precisions with brevity penalty.
    """
    precisions = []
    for n in range(1, max_n + 1):
        p = ngram_precision(reference, hypothesis, n)
        precisions.append(math.log(p) if p > 0 else float('-inf'))

    if float('-inf') in precisions:
        return 0.0
```

```python

    bp = min(1.0, math.exp(
        1 - len(reference.split()) / max(len(hypothesis.split()), 1)
    ))
    return bp * math.exp(sum(precisions) / max_n)


def tokenized_wer(reference: str, hypothesis: str) -> float:
    """WER using TextBlob tokenization (more consistent than split())."""
    ref_tokens = list(TextBlob(reference).words)
    hyp_tokens = list(TextBlob(hypothesis).words)
    ref_str = " ".join(ref_tokens)
    hyp_str = " ".join(hyp_tokens)
    return wer(ref_str, hyp_str)

# ── Main Evaluate Function ──────────────────────────────

def evaluate(raw_results, cleaned_results, ground_truth,
             output_dir, source_name, start_page):
    """
    Aligns OCR output pages to ground truth pages by page number,
    then computes all metrics.

    raw_results / cleaned_results : {image_path: text}
    ground_truth                  : {page_number: text}
    """
    print(f"\n{'='*60}")
    print(f"EVALUATION — {source_name}")
    print(f"{'='*60}")

    if not ground_truth:
        print("No ground truth available for this source, skipping eval.")
        return

    # ── Align pages ────────────────────────────────────────
    # image path has page number embedded: page_003.png → 3
    def page_num_from_path(path):
      base = os.path.basename(path)
      # Split column: page_003_left.png → "3_left"
      match = re.search(r'page_(\d+)_(left|right)', base)
      if match:
          return f"{int(match.group(1))}_{match.group(2)}"
      # Standard: page_003.png → 3
      match = re.search(r'page_(\d+)', base)
      return int(match.group(1)) if match else None

    gt_pages  = sorted(ground_truth.keys())
    raw_texts = []
    cln_texts = []
    gt_texts  = []

    for page_num in gt_pages:
        # Find corresponding OCR output path
        matching = [
            p for p in raw_results
            if page_num_from_path(p) == page_num
        ]
        if not matching:
            print(f"  WARNING: No OCR output found for page {page_num}, skipping.")
            continue

        path = matching[0]
        raw_texts.append(normalize_for_eval(raw_results[path]))
        cln_texts.append(normalize_for_eval(cleaned_results.get(path, raw_results[path])))
        gt_texts.append(normalize_for_eval(ground_truth[page_num]))

    if not gt_texts:
        print("No aligned pages found.")
        return

    gt_combined  = " ".join(gt_texts)
    raw_combined = " ".join(raw_texts)
    cln_combined = " ".join(cln_texts)

    # ── Alignment sanity check ──────────────────────────────
    gt_wc  = len(gt_combined.split())
```

```
    raw_wc = len(raw_combined.split())
    diff   = abs(gt_wc - raw_wc) / max(gt_wc, 1)
    print(f"\n  Ground truth words : {gt_wc}")
    print(f"  OCR output words   : {raw_wc}")
    if diff > 0.30:
        print(f"  WARNING: {diff:.0%} word count mismatch — check page alignment!")
    else:
        print(f"  Alignment OK ({diff:.0%} difference)")

    # ── CER / WER (jiwer) ──────────────────────────────────
    print(f"\n  --- CER / WER ---")
    print(f"  CER  before LLM : {cer(gt_combined, raw_combined):.2%}")
    print(f"  CER  after  LLM : {cer(gt_combined, cln_combined):.2%}")
    print(f"  WER  before LLM : {wer(gt_combined, raw_combined):.2%}")
    print(f"  WER  after  LLM : {wer(gt_combined, cln_combined):.2%}")

    # ── Tokenized WER (TextBlob) ───────────────────────────
    print(f"\n  --- Tokenized WER (TextBlob) ---")
    print(f"  before LLM : {tokenized_wer(gt_combined, raw_combined):.2%}")
    print(f"  after  LLM : {tokenized_wer(gt_combined, cln_combined):.2%}")

    # ── Individual n-gram precisions ───────────────────────
    print(f"\n  --- N-gram Precisions ---")
    for n in range(1, 5):
        r = ngram_precision(gt_combined, raw_combined, n)
        c = ngram_precision(gt_combined, cln_combined, n)
        print(f"  {n}-gram | before: {r:.2%}  after: {c:.2%}")

    # ── Save full eval report ──────────────────────────────
    report = f"""EVALUATION REPORT — {source_name}
{'='*60}

ALIGNMENT
  Ground truth words : {gt_wc}
  OCR output words   : {raw_wc}
  Difference         : {diff:.2%}

CER / WER
  CER before LLM : {cer(gt_combined, raw_combined):.2%}
  CER after  LLM : {cer(gt_combined, cln_combined):.2%}
  WER before LLM : {wer(gt_combined, raw_combined):.2%}
  WER after  LLM : {wer(gt_combined, cln_combined):.2%}

TOKENIZED WER (TextBlob)
  before LLM : {tokenized_wer(gt_combined, raw_combined):.2%}
  after  LLM : {tokenized_wer(gt_combined, cln_combined):.2%}

N-GRAM PRECISIONS
  1-gram | before: {ngram_precision(gt_combined, raw_combined, 1):.2%}  after: {ngram_precision(gt_combined, c
  2-gram | before: {ngram_precision(gt_combined, raw_combined, 2):.2%}  after: {ngram_precision(gt_combined, c
  3-gram | before: {ngram_precision(gt_combined, raw_combined, 3):.2%}  after: {ngram_precision(gt_combined, c
  4-gram | before: {ngram_precision(gt_combined, raw_combined, 4):.2%}  after: {ngram_precision(gt_combined, c
"""
    report_path = f"{output_dir}eval_report.txt"
    with open(report_path, "w", encoding="utf-8") as f:
        f.write(report)
    print(f"\n  Report saved → {report_path}")
```

Final Block 8 — Run Everything

```
# Step 1: OCR all sources (eval completely separate)
all_results = run_ocr_all(SOURCES)
```

```
============================================================
SOURCE : Buendia - Instruccion
Pages  : 2 → 4
============================================================
  Page 2 is too large. Scaling down to safe RAM limits.
  Page 2 → single page
  Page 3 is too large. Scaling down to safe RAM limits.
  Page 3 → single page
  Page 4 is too large. Scaling down to safe RAM limits.
  Page 4 → single page
```

```
  OCR: page_002.png
  OCR: page_003.png
  OCR: page_004.png

  OCR complete — 3 pages.
  Batch 1: quality 0.30 — skipping Groq, OCR already clean
  OCR outputs saved to outputs/Buendia - Instruccion/

============================================================
SOURCE : Covarrubias - Tesoro lengua
Pages  : 7 → 9
============================================================
  Page 7 → single page
  Page 8 → single page
  Page 9 → single page
  OCR: page_007.png
  OCR: page_008.png
  OCR: page_009.png

  OCR complete — 3 pages.
  Batch 1: quality 0.31 — skipping Groq, OCR already clean
  OCR outputs saved to outputs/Covarrubias - Tesoro lengua/

============================================================
SOURCE : Guardiola - Tratado nobleza
Pages  : 12 → 14
============================================================
  Page 12 → single page
  Page 13 → single page
  Page 14 → single page
  OCR: page_012.png
  OCR: page_013.png
  OCR: page_014.png

  OCR complete — 3 pages.
  Batch 1: quality 0.26 — skipping Groq, OCR already clean
  OCR outputs saved to outputs/Guardiola - Tratado nobleza/

============================================================
SOURCE : PORCONES.228.38 – 1646
Pages  : 1 → 5
============================================================
  Page 1 is too large. Scaling down to safe RAM limits.
  Page 1 → single page
  Page 2 is too large. Scaling down to safe RAM limits.
  Page 2 → single page
  Page 3 is too large. Scaling down to safe RAM limits.
```

```
run_eval_all(all_results, SOURCES)
EVALUATION  Buendia - Instruccion
============================================================

  Ground truth words : 609
  OCR output words   : 713
  Alignment OK (17% difference)

  --- CER / WER ---
  CER  before LLM : 48.36%
  CER  after  LLM : 48.36%
  WER  before LLM : 77.18%
  WER  after  LLM : 77.18%

  --- Tokenized WER (TextBlob) ---
  before LLM : 63.16%
  after  LLM : 63.16%

  --- N-gram Precisions ---
  1-gram | before: 78.75%  after: 78.75%
  2-gram | before: 60.03%  after: 60.03%
  3-gram | before: 44.79%  after: 44.79%
  4-gram | before: 32.10%  after: 32.10%

  Report saved → outputs/Buendia - Instruccion/eval_report.txt
  Auto-detected marker format: 'PDF\s*p(\d+)' from 'PDF p7'
  Parsed 'Covarrubias - Tesoro lengua transcription.docx': pages ['7', '8']

============================================================
```

```
  Alignment OK (2% difference)

  --- CER / WER ---
  CER  before LLM : 9.44%
  CER  after  LLM : 9.44%
  WER  before LLM : 42.80%
  WER  after  LLM : 42.80%

  --- Tokenized WER (TextBlob) ---
  before LLM : 35.35%
  after  LLM : 35.35%

  --- N-gram Precisions ---
  1-gram | before: 71.28%  after: 71.28%
  2-gram | before: 51.44%  after: 51.44%
  3-gram | before: 36.89%  after: 36.89%
  4-gram | before: 25.19%  after: 25.19%

  Report saved → outputs/Covarrubias - Tesoro lengua/eval_report.txt
  Auto-detected marker format: 'PDF\s*p(\d+)' from 'PDF p12'
  Parsed 'Guardiola - Tratado nobleza transcription.docx': pages ['12', '13', '14']

============================================================
EVALUATION — Guardiola - Tratado nobleza
============================================================
```